

Working With Nordic Semiconductor SoCs

1 Introduction

Nordic Semiconductor is a Norwegian fabless semiconductor company that specialises in wireless communication technology. It is most commonly known for its award-winning Bluetooth Low Energy solutions characterised by high performance, ultra-low power, and ease of design. The System on Chips (SoCs) by Nordic Semiconductor have been used by a considerable number of distinguished brands across the world for projects in healthcare, gaming, mobile phone accessory, toys, sports and fitness, and wireless PC peripherals.[1]

2 The NRF Connect SDK:

The NRF Connect SDK is the software development kit provided by Nordic Semiconductor to work with their SoCs. Having a reputation for providing low-power wireless applications, the SDK is capable of building highly optimized software for memory-constrained devices and is integrated with four main repositories: including the Zephyr Real-Time Operating System (RTOS), MCU-Boot, nrfxlib, and nrf. The strength of the SDK is in having a single code base for all the Nordic Semiconductor devices, making it platform-agnostic. In other words, the source code is highly decoupled, allowing the same application code to perform correctly on multiple different hardware platforms and configurations with minimal changes. After creating an application source code, based on the C/C++ language, it can be compiled using the Kconfig, Device Tree, Cmake, Ninja, and GCC toolchains.

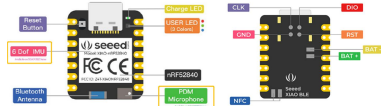


Figure 1: The Seeed Studio Xiao nRF52840

Alternatively, the Nordic Semiconductor SoCs are not limited to only the NRF Connect SDK. It is very common for these SoCs, like the nRF52 series, to be programmed using the Arduino IDE. The compatibility of programming languages and RTOS that can flash the device depends on the bootloader installed. Devices like the Seeed Studio XIAO nRF52840 Sense (see Figure 1) can be programmed with NuTTX (RTOS), Zephyr (RTOS), CircuitPython, and Arduino.[7]

The recommended programming and development approach would be to use Arduino or CircuitPython for more rapid prototyping purposes; the former can be implemented using the Arduino IDE, or preferably Platform IO which can be installed as a Visual Studio (VS) Code extension. For more complex and resource-constrained projects, it is preferable to use a RTOS like NuTTX, Zephyr or at best the NRF Connect SDK, which is also available as an extension in VS Code.

Software Development Kits (SDKs): A collection of tools provided by manufacturers that include libraries, sample code, and support for the operating systems that have been tested and optimized for the hardware.

Toolchains: Consists of compilers, linkers, and debuggers capable of translating high-level code into machine code that the processor can execute.

Bootloader compatibility: Responsible for loading the operating system or other software into memory, the bootloader determines which firmware can be flashed to the device, e.g., To flash Arduino code, a bootloader that can recognize and execute Arduino-compiled binaries must be installed. A RTOS or programming language can be flashed to the hardware only if it supports the processor being used, e.g., the nRF52840 is based on the ARM Cortex-M4 Processor.[4]

3 The Development Process

The developmental process of working with Nordic Semiconductor chips requires a structured approach to go from the planning phase, to testing and prototyping, to the full-scale realisation. Below are the advised steps to follow when creating your own projects followed by an example:

1. Provide a clear project definition
2. Breakdown the goal into sub-tasks
3. Look for implementation examples
4. Verify feasibility and compatibility of examples
5. Full-scale integration with incremental testing

Project Definition: A clearly defined project lists the requirements, impediments and objectives of the project e.g., A medical-grade accurate continuous data collection and transmission system that samples a high-frequency input signal and transmits this over wireless communication

Task Breakdown: Given the goal previously defined, the tasks required to achieve this are outlined e.g., data collection would require an Analogue-to-Digital Converter (ADC), and Bluetooth Low Energy can be used for providing wireless communication

Code Examples: Look for code examples and implementations starting with Nordic Semiconductor's Github. [3][5] There are likely multiple implementations for the same process/task meaning it would be prudent to sift through the different options e.g., the ADC can be configured to work with the Zephyr API or the nrfx SAADC drivers, the first improves portability and power management, the other allows higher sampling rates and better precision. In this case, the latter would be favoured.

Independent Testing of Subtasks: Having gathered all the possible code examples and implementations, testing each one for its feasibility and compatibility in the project is important e.g., the ADC and BLE subtask codes must be able to perform their tasks individually. This requires building and flashing the code then subjecting it checks to verify its functionality

Integration and System Testing: There are likely to be many problems when performing integration. However, a step-by-step process with many incremental tests are a good way to minimise or remove errors from entering the code.

Summary: Due to the highly decoupled source code available with the NRF Connect SDK, understanding what the API functions do can be complicated and tedious. It is therefore important to be able to navigate the technical documentation provided by or used by Nordic Semiconductor. [6]

4 Recommendations and Best Practices

Take courses: The Nordic Semiconductor chips are typically used in Internet of Things (IoT) projects. A valuable resource to equip fundamental theoretical knowledge and hands-on skills is the Nordic Developer Academy courses.[2] Currently, they offer five short courses that provide a certificate upon completion.

1. nRF Connect SDK Fundamentals
2. Wi-Fi Fundamentals
3. Bluetooth Low Energy Fundamentals
4. Cellular IoT Fundamentals
5. nRF Connect SDK Intermediate

These courses are tailored for beginners who have no prior experience with Nordic Semiconductor and/or IoT projects, but do know how to code in C/C++. The nRF Connect SDK Fundamentals course provides tutorials from the absolute basics of installing the nRF Connect SDK with Visual Studio Integrated Development Environment (VS Code IDE), and follows up with how to build and flash simple nRF Connect applications. The Bluetooth Low Energy Fundamentals course teaches developers how to customize their BLE protocol to suit their applications, exploring into different network topologies, data exchange protocols, and security levels.

Use Github It is a great idea to use GitHub with these projects to have a version history of all the working code. All of these can be committed and pushed directly from the source control icon located in the left tab of VS Code. It is also useful to maintain a repository of all the sample codes that have been used for easy reference. Sample codes can also be cloned directly from VS Code by using 'Git: Clone'.[8]

nRF Connect for Mobile: When developing BLE applications, it is very helpful to use the nRF Connect for Mobile app (See Figure 2) which is packed with many useful monitoring abilities for the protocol itself, and also of the wireless signal. However, it should be noted that the app differs when using IOS or Android; the former having greater functionality as well as providing a more visually pleasing GUI.

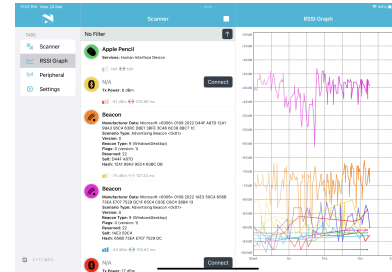


Figure 2: The NRF Connect for Mobile App

5 Conclusion

This short article explores the usage of the powerful Nordic Semiconductor SoCs for product development. These chips are distinguished by their advanced BLE solutions and are therefore favorable in numerous IoT applications across a broad spectrum of projects. The SoCs can be programmed using the NRF Connect SDK, and can often support other programming languages and RTOS like Arduino, MicroPython, and NuTTX RTOS, depending on manufacturer support. The development process has been outlined and resources were given to immediately examine a range of sample codes. Finally, recommendations and best practices were given based on the experience of other developers.

6 References

- [1] Nordic Semiconductor. *About Nordic Semiconductor*. Online. URL: <https://www.nordicsemi.com/About-us>.
- [2] Nordic Semiconductor. *DevAcademy*. Online. URL: <https://academy.nordicsemi.com/>.
- [3] Nordic Semiconductor. *NordicDeveloperAcademy*. Online. URL: <https://github.com/NordicDeveloperAcademy>.
- [4] Nordic Semiconductor. *nRF52840*. Online. URL: <https://www.nordicsemi.com/Products/nRF52840>.
- [5] Nordic Semiconductor. *sdk-nrf*. Online. URL: <https://github.com/nrfconnect/sdk-nrf>.
- [6] Nordic Semiconductor. *TechDocs*. Online. URL: <https://docs.nordicsemi.com/>.
- [7] Seeed Studio. *Getting Started with Seeed Studio XIAO nRF52840 Series*. Online. URL: https://wiki.seeedstudio.com/XIAO_BLE/.
- [8] Visual Studio Code. *Working with Github in VS Code*. Online. URL: <https://code.visualstudio.com/docs/sourcecontrol/github>.